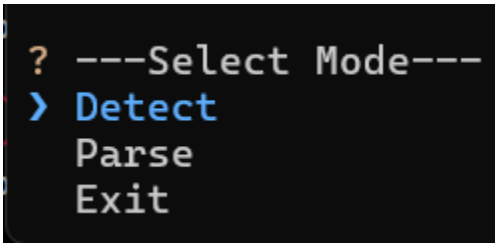
PickyRTL

Project Description

PickyRTL is a static analysis framework for detecting hardware vulnerabilities associated with CWEs. The detection tool allows users to parse HDL files and analyze them for vulnerabilities. The tool detects vulnerabilities associated with CWE-1245, CWE-1233, CWE-226, and CWE-1431.

User Interface Specification

The user interface for PickyRTL is a command line interface.



Test Plan and Results

Test Plan

The overall testing approach focuses on verifying correct program functionality rather than validating the absolute correctness of the detection algorithms, since their outputs cannot be predicted with complete certainty. Testing will therefore emphasize functional behavior across the full execution pipeline, including program startup, file parsing, detection execution, and result storage. These tests are designed to ensure that the system reliably accepts supported input files, successfully performs detection processing, and correctly saves outputs for further analysis. Together, this coverage provides confidence that the program operates as intended and delivers a usable and dependable workflow for end users

Test Case Description

1. **Startup-1**
2. This test will ensure that on program startup the user will be presented with the correct options
3. Start the program and look at the options on screen
4. Command to start the program
5. Three options show up in the command line: “Parse”, “Detect”, and “Exit”
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Parse-1**
2. This test will ensure that when the user selects “Parse” it will bring up a file explorer to select a folder/file for parsing
3. Select the “Parse” option and traverse through the folders
4. No inputs
5. A file explorer will populate in the command line
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Parse-2**
2. This test will ensure that when the user selects a file for parsing it will correctly parse the HDL file into a JSON AST
3. Select a file to parse and check that the parsed file was saved
4. HDL file
5. The file will be parsed and the saved file location will be output to the command line
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Parse-3**
2. This test will ensure that when the user selects a folder for parsing it will parse all of the files within the folder into JSON ASTs
3. Select a folder to parse and check that each file within the folder was parsed and saved
4. Folder with more than one HDL files in it
5. The files within the folder will be parsed and the saved file locations will be output to the command line
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Detection-1**
2. This test will ensure that when the user selects “Detect” it will bring up a file explorer to select a folder/file for detection
3. Select the “Detect” option and traverse through the folders
4. No inputs
5. A file explorer will populate in the command line
6. Normal

7. Blackbox
8. Functional
9. Unit

1. **Detection-2**
2. This test will ensure that when the user selects a file for detection it will run detection on the parsed HDL file
3. The user will select an HDL to run detection on and ensure the detection starts
4. HDL file
5. The detection algorithm starts
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Detection-3**
2. This test will ensure that when the user selects a folder for detection it will run detection all of the HDL files within the folder
3. The user will select a folder that contains more than one HDL file and select it to run detection on
4. Folder with more than one HDL file
5. The detection algorithm starts
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Matching-1**
2. This test will ensure that when the user selects “Fuzzy” matching they will not be able to enter a number greater than 100
3. The user will select “Fuzzy” and try to input a number larger than 100
4. Any number larger than 100
5. The number will reset to 100
6. Abnormal
7. Blackbox
8. Functional
9. Unit

1. **Matching-2**

2. This test will ensure that when the user selects “Fuzzy” matching they will not be able to enter a number less than 0
3. The user will select “Fuzzy” and try to input a number less than 0
4. Any number less than 0
5. The number will reset to 0
6. Abnormal
7. Blackbox
8. Functional
9. Unit

1. **Save-1**
2. This test will ensure that when the user selects “Yes” to saving the results, it will bring up a file explorer and the user can select a folder to save the results in
3. The user will select “Yes” when prompted if they want to save the results and traverse through the file explorer
4. No inputs
5. A file explorer will populate in the command line
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Save-2**
2. This test will ensure that when the user selects a save location it will prompt the user for a file name
3. The user will select a save location for the results file and enter a name when prompted
4. Location to save results and file name
5. User is prompted for results file name
6. Normal
7. Blackbox
8. Functional
9. Unit

1. **Save-3**
2. This test will ensure that when results are saved they are actually saved
3. The user will save the results and check that the file was saved in the selected location
4. Save file location
5. The file will be saved in the correct location and with the correct file name
6. Normal
7. Blackbox
8. Functional
9. Unit

Test Case Matrix

Test Case ID	Normal/ Abnormal	Blackbox/ Whitebox	Functional/ Performance	Unit/ Integration
Startup-1	Normal	Blackbox	Functional	Unit
Parse-1	Normal	Blackbox	Functional	Unit
Parse-2	Normal	Blackbox	Functional	Unit
Parse-3	Normal	Blackbox	Functional	Unit
Detection-1	Normal	Blackbox	Functional	Unit
Detection-2	Normal	Blackbox	Functional	Unit
Detection-3	Normal	Blackbox	Functional	Unit
Matching-1	Abnormal	Blackbox	Functional	Unit
Matching-2	Abnormal	Blackbox	Functional	Unit
Save-1	Normal	Blackbox	Functional	Unit
Save-2	Normal	Blackbox	Functional	Unit
Save-3	Normal	Blackbox	Functional	Unit

User Manual

Requirements

- Ubuntu / WSL (tested on Ubuntu 22.04)
- Python 3.10+

Installation

1. git clone https://github.com/UCdasec/PickyRTL.git

Running the Program

1. Change into the /Detection directory
2. cd Detection
3. To run the program execute ./run_program.sh

Parsing an HDL File

1. Start the program
2. Use the arrow keys to navigate to Parse and press "Enter"
3. ---Select Mode---
4. Detect
   - Parse
5. Exit
6. Use the arrow keys to navigate the file explorer and select an HDL file or a folder of HDL files
7. The parsed files will be saved under PickyRTL/Detection/Parsed_files

Running Detection

1. Start the program

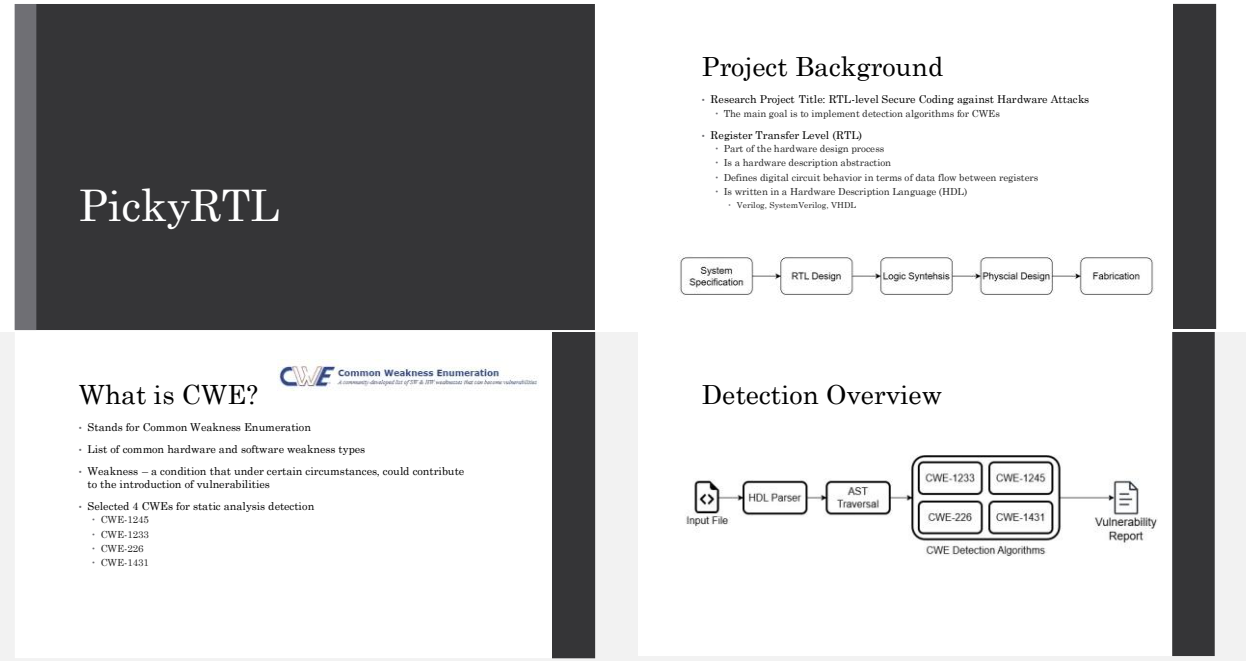
2. Use the arrow keys to navigate to Detect and press "Enter"
3. ---Select Mode---
    - ➢ Detect
4. Parse
5. Exit
6. Use the arrow keys to navigate the file explorer and select a parsed file or a folder of parsed files
7. When detection is finished use the arrow keys to select a save location under PickyRTL/Detection/Results directory
8. Enter a name for the file and press "Enter"

FAQs

1. What CWEs does this tool focus on?

PickyRTL detects weaknesses for CWE-226, CWE-1233, CWE-1245, and CWE-1431

Spring Final PPT Presentation



CWE-1245: Improper Finite State Machines (FSMs) in Hardware Logic

- FSMs are commonly used in hardware logic for secure data operations, security state, and other functionality
- Vulnerable Patterns
  - Deadlocks
  - Unreachable states
  - Incomplete state coverage
- Consequences
  - System could enter a deadlocked state
  - Bypass security authentication
  - Unpredictable system behavior
- Detection
  1. Incomplete state coverage?
  2. Unreachable states?
  3. Deadlock states?

```
module fsm(input clk, input [1:0] user_input, output reg
    [1:0] state);
    always @(posedge clk) begin
        case (user_input):
            3'h0: state = 2'h3;
            3'h1: state = 2'h2;
            3'h2: state = 2'h1;
            default: state = 2'h0;
        endcase
    end
endmodule
```

CWE-1233: Security Sensitive Hardware Controls with Missing Lock Bit Protection

- Lock Bit – A hardware mechanism that prevents modification of registers after they have been set
- Security-critical registers should be protected with lock bits
- Consequences
  - Security policies could be bypassed if access controls are modified
  - Keys could be overwritten
- Detection
  1. Identify the lock bit
  2. Gather security-critical registers
  3. Are assignments made to the registers unprotected?

```
module unprotected_key(input clk, input rst_ni, input
    reglk_ctrl_i, input [31:0] data_i);
    logic [31:0] key0;
    always @(posedge clk) begin
        key0 <= reglk_ctrl_i ? key0 : data_i;
    end
endmodule
```

CWE-226: Sensitive Information in Resource Not Removed Before Reuse

- Registers often store sensitive information such as cryptographic keys, authentication tokens, etc.
- If the registers aren't cleared after being used, the data persists and can be accessed by subsequent operations
- Consequences
  - Cryptographic key leakage to unauthorized process
  - Sensitive data could be accessed by untrusted parties
- Detection
  1. Identify the reset signal
  2. Gather registers that need to be reset
  3. Are the registers properly reset?

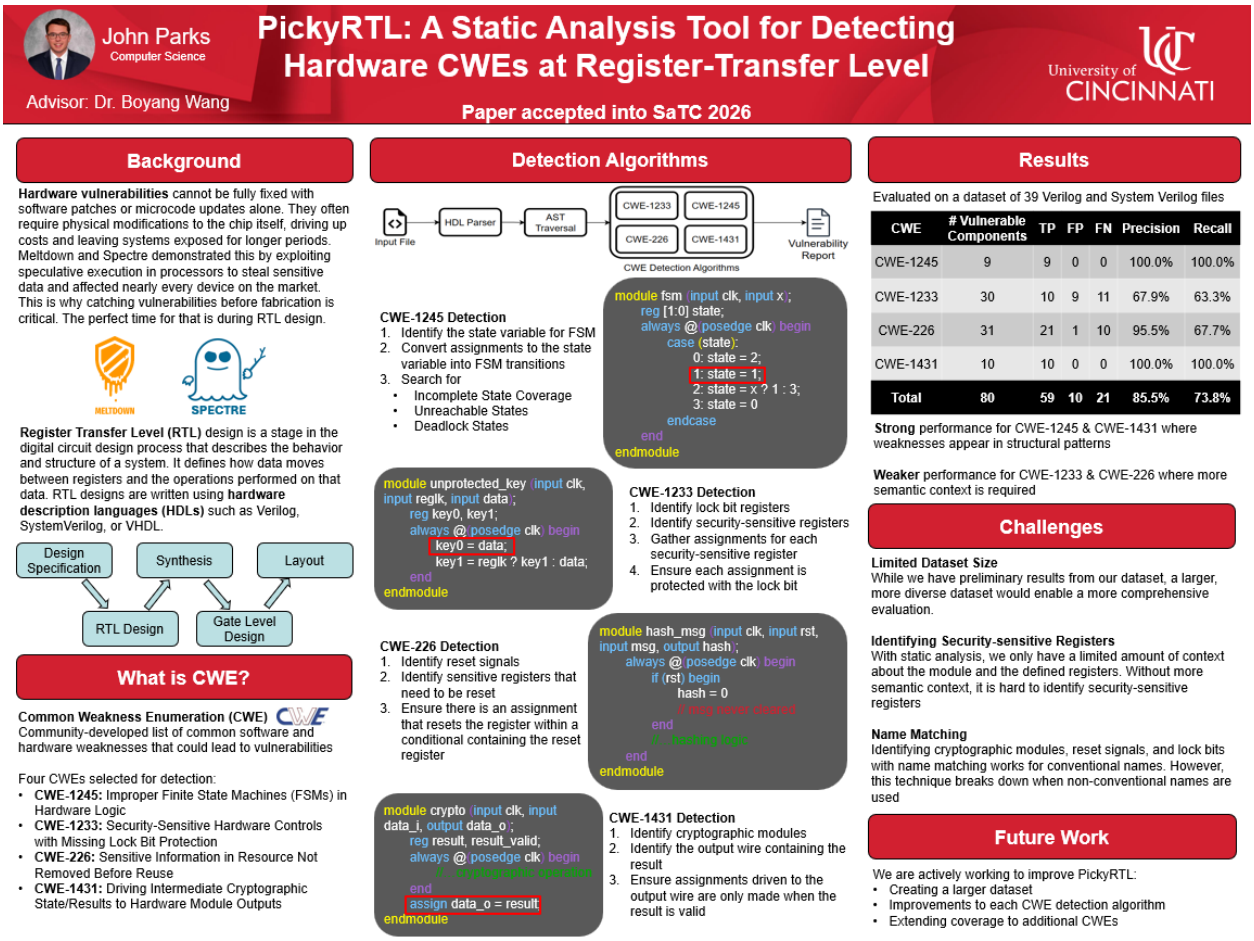
```
module hash_msg(input clk, input rst_ni);
    reg [2047:0] msg_out, msg_in;
    always @(posedge clk) begin
        if(!rst_ni) begin
            msg_out <= 2048'b0;
            msg_in <= 2048'b0;
        end
        // .. (omitted hashing logic)
    end
endmodule
```

CWE-1431: Driving Intermediate Cryptographic State/Results to Hardware Module Outputs

- Cryptographic modules often involve intermediate results between rounds of encryption
- These results should remain internal and not be exposed to output ports
- Consequences
  - Intermediate results could be observed by untrusted parties through side channel attacks
- Detection
  1. Identify the cryptographic module output
  2. Find any registers containing intermediate results
  3. Are intermediate results assigned to the module output before checking if the result is valid?

```
module crypto_core(input clk, input data_i, output data_o,
    output result_valid);
    logic result;
    always @(posedge clk) begin
        //.... (omitted cryptographic algorithm logic)
    end
    assign data_o = (result_valid) ? result : 0;
endmodule
```

Final Expo Poster



Assessments

- ## Initial Self-Assessment

My senior design project, RTL-Level Secure Coding Against Hardware Attacks, is a continuation of the research I have been working on over the past summer. The project focuses on identifying vulnerabilities in hardware design at the register-transfer level (RTL) in order to protect systems against potential attacks. From my academic perspective, this work is about applying computer science principles I have learned in my coursework to detect unsecure code patterns and help create secure patterns before a hardware design is finalized. Some of my roles include developing detection algorithms, collecting datasets, and writing technical reports and research papers. My goal is to make hardware designs more resilient to hardware attacks as new methods for attacks are being discovered. Protecting against hardware attacks is just as important as protecting against software attacks because they can be just as damaging.

My college coursework has provided me with both the technical and communication skills necessary to contribute effectively to my senior design project. The course Intro to Algorithms gave

me experience in designing and implementing efficient algorithms, which is essential when analyzing large hardware designs. Also, Technical Writing strengthened my ability to convey technical information and data clearly through reports and other documentation. This directly connects to the reports I have to write on my work and hopefully a research paper. My work in Artificial Intelligence courses emphasized the importance of a high-quality dataset to improve the accuracy of results. While I may not be training models, having a wide range of data for vulnerability detection will be paramount to improving the accuracy of my detection algorithms. Altogether, these courses have given me strong foundation in computer science principles which will guide me throughout the development of my project.

My co-op experiences have also played a key role in preparing me for this senior design project. Most directly, my most recent co-op was the beginning of this research, which got me started with hardware designs, vulnerabilities, and detection algorithms. This knowledge will obviously be very helpful for my work down the road. Before that, I conducted research in Spain which gave me a deeper understanding of the research process, even though the project focus was different. Both of these experiences have taught me how to approach open-ended problems and adapt to new challenges. My earlier co-ops have strengthened my coding practices and improved my critical thinking in solving complex problems. The combination of all my co-op experiences has formed me into a more capable programmer and a more effective problem-solver, which will be essential for this project.

I am motivated to work on this project because of the importance of secure hardware in protecting private data. As hardware attacks become more sophisticated, it is critical that designs are built securely from the beginning. I believe secure hardware is just as important as secure software in the role of safeguarding sensitive information. The initial approach I have taken is designing a static analysis tool that can scan hardware designs for patterns connected to known vulnerabilities. This method should provide an effective defense by flagging insecure code patterns early in the design process. In the future, I am also interested in exploring other techniques, such as using LLMs to enhance detection capabilities.

The expected result of my project is the creation of a tool that can be used to evaluate hardware designs for specific vulnerabilities at the RTL level. In addition to this, I also hope to write a research paper that summarizes my methods, findings, and impact of the work. To evaluate my contributions, I will measure the effectiveness of the tool through vulnerability detection accuracy. I will also reflect on the quality of my written contributions, including any research papers, ensuring that my work was clearly communicated. Throughout the project, I will work with my project advisor to determine the significance of the work I have accomplished. Using technical and academic performance as measures, I think I will be able to judge whether I have met my goals.

- ## Final Self-Assessment

My senior design project, PickyRTL, ended with the creation a fully functional static analysis tool that automatically detects hardware Common Weakness Enumerations (CWEs) in Register-Transfer Level (RTL) code. The project started as a research co-op I began during the summer semester, and this year I took that work and built it into a complete tool that was accepted for publication at a peer-reviewed conference. My responsibilities encompassed every stage of the project from researching

related work, designing detection algorithms, collecting a dataset of 39 HDL files, evaluating the tool, and writing the final research paper.

The skills I identified in my initial fall assessment proved directly applicable throughout the project, and in each step I built on top of them. My background in algorithms was essential for designing the AST traversal and CWE-specific detection logic, particularly the state-coverage, unreachable-state, and deadlock checks for CWE-1245, which required graph reasoning. My Technical Writing coursework paid off when writing a conference paper, where clarity in communicating my methods was just as important as the technical work itself. The emphasis on high-quality datasets and proper evaluation from my AI coursework shaped my approach to evaluating the tool. Rather than testing on a small or homogeneous sample, I deliberately assembled examples from multiple sources including the CWE MITRE website, HACK@DAC competition repositories, and the VUL-FSM dataset, totaling over 8,900 lines of code. One of the best accomplishments of the tool was achieving 100% precision and recall for CWE-1245 and CWE-1431 on the gathered dataset. These results demonstrated that static analysis works well for weaknesses that occur in structural. The most significant obstacle I faced was identifying security-sensitive registers for CWE-1233 and CWE-226 without full design context, which led to lower precision and recall for those CWEs. Working through that limitation deepened my understanding of the gap between rule-based static analysis and the richer semantic knowledge a human reviewer would bring to the same code.

Summary of Hours and Justification

I worked ~20-24 hours every week on the project.

Summary of Expenses

No expenses to date

Appendix

Commit Link